TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN NHẬT HUY – 522H0134
HUỲNH NHẬT HUY – 523C0012
NGUYỄN THỊ BẢO TRÂN - 52200089

ỨNG DỤNG REINFORCEMENT LEARNING TRONG TỐI ƯU THUẬT TOÁN DI TRUYỀN THÍCH NGHI CHO BÀI TOÁN ĐỊNH TUYẾN XE CÓ RÀNG BUỘC THỜI GIAN (RL-GA CHO VRPTW)

BÁO CÁO TIẾN ĐỘ NGHIÊN CỨU KHOA HỌC

Người hướng dẫn
TS. Hồ Thị Linh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

Các Công Việc Đã Hoàn Thành (Tháng 10 & 11/2025)

Hoàn thành Framework Code: Đã thiết lập hoàn chỉnh framework cho cả **Baseline Adaptive ALNS** và **DQN-ALNS** trên môi trường Python/PyTorch.

Xử lý Dữ liệu: Đã xây dựng bộ đọc (parser), có khả năng xử lý cả hai bộ dữ liệu benchmark tiêu chuẩn: **Solomon** và **Gehring-Homberger** (đã trực quan hóa phân bố các loại bài toán C, R, RC).

Huấn luyện Mô hình Học Tăng cường: Đã triển khai và huấn luyện thành công tác nhân **DQN Agent** để học chính sách lựa chọn toán tử phá vỡ (Destroy Operator) trong chu trình ALNS, thay thế cho cơ chế Adaptive Weighting cổ điển.

Đánh giá So sánh: Đã chạy thử nghiệm và so sánh hiệu suất giữa **Baseline ALNS** và **DQN-ALNS** trên bộ dữ liệu đại diện gồm 12 instances (6 Solomon + 6 Homberger), tập trung vào các tiêu chí: **Chi phí**, **Số lượng xe**, và **Thời gian chạy**.

1. Thu Thập và Xử Lý Dữ Liệu Benchmark (Hoàn thành 15% tiến độ)

- **Tải và Phân tích Dữ liệu:** Đã tải xuống và phân tích cấu trúc của bộ dữ liệu tiêu chuẩn **Solomon VRPTW benchmark**.
- **Lựa chọn Instance ban đầu:** Đã chọn các bộ dữ liệu nhỏ và vừa (ví dụ: **C101** - tập trung vào cụm khách hàng, **R101** - tập trung vào quãng đường phân tán) để làm dữ liệu huấn luyện và kiểm thử ban đầu.
- **Xử lý Dữ liệu:** Xây dựng module đọc và chuyển đổi định dạng dữ liệu Solomon sang cấu trúc dễ dàng xử lý bằng Python, chuẩn bị cho quá trình cài đặt thuật toán.

2. Chi Tiết Triển Khai (DQN-ALNS)

Thành phần	**Triển khai**	**Chi tiết**
Mô hình Cơ bản	Adaptive Large Neighborhood Search (ALNS)	Sử dụng các toán tử: random_removal, route_removal (Destroy) và greedy_insertion (Repair).
Môi trường RL	Deep Q-Learning (DQN)	Agent quyết định lựa chọn toán tử Destroy nào ở mỗi bước lặp.
Trạng Thái (State)	Vector 2 chiều	Sự khác biệt chi phí hiện tại so với tốt nhất và mức độ "bế tắc" (số lần lặp chưa cải thiện).
Action	2 hành động rời rạc	Chọn random_removal (Phá nhỏ) Chọn route_removal (Phá lớn)
Reward	Reward Function	(Global Best), (Local Improvement), (Chấp nhận SA).
Công cụ ML	PyTorch (QNetwork)	Sử dụng 2 mạng (Policy Net và Target Net) và bộ nhớ ReplayMemory. Đã lưu model dưới định dạng .safetensor.

I. BÁO CÁO TIẾN ĐỘ & KẾT QUẢ THỰC NGHIỆM NCKH

1. Mục Tiêu Đã Đạt Được

- **Hoàn thành Framework Code:** Đã thiết lập hoàn chỉnh framework cho cả **Baseline Adaptive ALNS** và **DQN-ALNS** trên môi trường Python/PyTorch.
- **Xử lý Dữ liệu:** Đã xây dựng bộ đọc (parser) mạnh mẽ, có khả năng xử lý cả hai bộ dữ liệu benchmark tiêu chuẩn: **Solomon** và **Gehring-Homberger** (đã trực quan hóa phân bố các loại bài toán C, R, RC).
- **Huấn luyện Mô hình Học Tăng cường:** Đã triển khai và huấn luyện thành công tác nhân **DQN Agent** để học chính sách lựa chọn toán tử phá vỡ (Destroy Operator) trong chu trình ALNS, thay thế cho cơ chế Adaptive Weighting cổ điển.
- **Đánh giá So sánh:** Đã chạy thử nghiệm và so sánh hiệu suất giữa **Baseline ALNS** và **DQN-ALNS** trên bộ dữ liệu đại diện gồm 12 instances (6 Solomon + 6 Homberger), tập trung vào các tiêu chí: **Chi phí**, **Số lượng xe**, và **Thời gian chạy**.

2. Chi Tiết Triển Khai (DQN-ALNS)

Mô hình Cơ bản	Adaptive Large Neighborhood Search (ALNS)	Sử dụng các toán tử: random_removal, route_removal (Destroy) và greedy_insertion (Repair).
Môi trường RL	Deep Q-Learning (DQN)	Agent quyết định lựa chọn toán tử Destroy nào ở mỗi bước lặp.
State	Vector 2 chiều	: Sự khác biệt chi phí hiện tại so với tốt nhất và mức độ "bế tắc" (số lần lặp chưa cải thiện).
Action	2 hành động rời rạc	Chọn random_removal (Phá nhỏ) Chọn route_removal (Phá lớn)
Reward	Reward Function	(Global Best), (Local Improvement), (Chấp nhận SA).
Công cụ ML	PyTorch (QNetwork)	Sử dụng 2 mạng (Policy Net và Target Net) và bộ nhớ ReplayMemory. Đã lưu model dưới định dạng .safetensor.

3. Kết Quả Thực Nghiệm và Phân Tích

Metric	Baseline ALNS	DQN-ALNS	Cải thiện Trung bình (TB)
Cost	Tổng hợp (12 files)	Tổng hợp (12 files)	**+0.50%** (TB tổng thể)

Thời gian chạy (s)	giây	giây	**Tốc độ nhanh hơn 9%**
Số xe	Tổng hợp	Tổng hợp	**DQN giúp giảm số xe** ở một số bài (ví dụ: c201 giảm từ 4 3 xe).

Phân Tích Hiệu suất Tối ưu (Chi phí)

- **Cải thiện Nổi bật:** DQN đạt hiệu suất vượt trội trên các bài toán **Clustered (C)**, đặc biệt là 200_C1_2_1 (**+14.52%**) và c201 (**+7.21%**).
- **Phát hiện:** *Khi DQN tìm thấy giải pháp tốt hơn, mức cải thiện rất đáng kể (TB **+4.91%**), chứng tỏ DQN có khả năng thoát khỏi các tối ưu cục bộ (Local Optimum) mà Adaptive Weighting cổ điển bị mắc kẹt.*
- **Trái chiều:** DQN hoạt động kém hơn trên một số bài toán **Random (R)** và **Random+Clustered (RC)**, cho thấy Agent cần được huấn luyện sâu hơn hoặc cần một định nghĩa State/Reward phức tạp hơn để xử lý sự phân tán của dữ liệu R.

Phân Tích Thời gian Chạy (Tốc độ)

- **Phát hiện bất ngờ:** DQN-ALNS chạy nhanh hơn Baseline trung bình **7.72 giây**.
- **Giả thuyết:** DQN học được **chính sách lựa chọn toán tử hiệu quả** hơn. Nó ưu tiên các toán tử có khả năng mang lại phần thưởng cao, tránh lãng phí thời gian vào các phép biến đổi không hiệu quả, dẫn đến tốc độ hội tụ nhanh hơn.

II. HƯỚNG PHÁT TRIỂN TIẾP THEO

Cải tiến State/Action Space:

- **State:** Bổ sung thêm các thông số như **độ phân tán của Time Window**, **Capacity Utilization** để DQN có cái nhìn toàn diện hơn về môi trường.
- **Action:** Mở rộng Action space để DQN không chỉ chọn toán tử Destroy mà còn học cách điều chỉnh tham số của toán tử đó (ví dụ: số lượng khách hàng cần loại bỏ - trong random_removal).

- **Tăng Cường Huấn luyện:** Huấn luyện DQN trên một tập hợp instances lớn hơn và đa dạng hơn (meta-learning) để Agent có khả năng tổng quát hóa (generalization) tốt hơn, đặc biệt với các bài toán loại R.